

ECMAScript 2015

ES6 Deep Dive

EGCI427 Web Programming — Lecture 05

01

What is ES6?

History, motivation, and compatibility

02

Variables: let & const

Block scoping vs var

03

Arrow Functions

Syntax and lexical this

04

Template Literals

String interpolation and multi-line

05

Destructuring

Arrays and objects

06

Default, Rest & Spread

Flexible function signatures

07

Classes

OOP syntax and inheritance

08

Modules

import / export

09

Promises & Async

Async programming

10

New Built-ins

Map, Set, Symbol, etc.

Published June 17, 2015 — the biggest update to JavaScript in years.



6-Year Gap

ES5 (2009) → ES6 (2015)
6 years of evolution



Safer Syntax

Fixes var scoping pitfalls
and this context confusion



Native Modules

Built-in import/export
no more script soup



Scale Ready

Classes, Promises, Maps
for large applications

ES6 Compatibility Table

<https://kangax.github.io/compat-table/es6/>

Transpiling: Babel converts ES6 → ES5/ES3 so older browsers can run modern code.

ES6 Source → Babel Transpiler → ES5 Output → All Browsers ✓

02 Variables

let & const

Block scoping — the most common ES6 fix

The Problem with var

❌ ES5 — var (function-scoped)

```
// ES5 — var has FUNCTION scope (not block scope!)
function foo() {
  var num = 1;
  if (num === 1) {
    var num = 2;      // ← SAME variable! overwrites!
    console.log(num); // 2
  }
  console.log(num);   // 2 ← Surprise!
}
foo();
```

✅ ES6 — let (block-scoped)

```
// ES6 — let has BLOCK scope
function foo() {
  let num = 1;
  if (num === 1) {
    let num = 2;      // ← different variable
    console.log(num); // 2
  }
  console.log(num);   // 1 ← Correct!
}
foo();
```

Loop Scope Difference

❌ ES5 — var loop

```
// var: leaks out of the loop
for (var i = 0; i < 3; i++) {
  console.log(i); // 0, 1, 2
}
console.log(i);   // 3 ← still accessible!
```

✅ ES6 — let loop

```
// let: scoped to the loop block
for (let i = 0; i < 3; i++) {
  console.log(i); // 0, 1, 2
}
console.log(i); // ReferenceError!
// i is not defined — correct!
```

const prevents reassignment, not deep mutation

const behaviour

```
// Primitive - cannot be reassigned
const PI = 3.14159;
PI = 3; // TypeError: Assignment to constant variable.
const PI = 3; // SyntaxError: Identifier already declared.

// Objects - reference is const, but properties ARE mutable
const user = { name: 'Alice', age: 20 };
user.age = 21; // ✓ OK - mutating a property
user.role = 'admin'; // ✓ OK - adding a property
user = { name: 'Bob' }; // ✗ TypeError - can't reassign reference

// Arrays - same rules
const nums = [1, 2, 3];
nums.push(4); // ✓ OK
nums = [5, 6, 7]; // ✗ TypeError
```

Key Rules

- Must be initialized at declaration
- Cannot be reassigned
- Block-scoped (like let)
- Object contents can still change
- Prefer const by default
- Use let only when reassignment is needed



Rule of Thumb: Always start with const. Only change to let if you need to reassign. Never use var in ES6+.



Identify the bug and rewrite using let/const correctly



Buggy ES5 Code

```
// Fix this ES5 code using ES6 let/const
// Task: print scores and calculate average
var scores = [90, 85, 78, 92, 88];
var total = 0;

for (var i = 0; i < scores.length; i++) {
  var score = scores[i];
  total += score;
}

var average = total / scores.length;
console.log("Average: " + average);
console.log("i is: " + i);    // Bug! i leaks out
console.log("score is: " + score); // Bug! also leaks
```



Fixed ES6 Code

Hints: • scores array never changes → const • total accumulates → let • loop variables stay inside → let/const

Try it in your browser console or Node.js REPL!

03 Arrow Functions

Shorter syntax + lexical this binding

Concise function expressions with implicit returns

Syntax Patterns

```
// ES5 regular function
function add(a, b) {
  return a + b;
}

// ES6 arrow function - full form
const add = (a, b) => {
  return a + b;
};

// ES6 - implicit return (no braces for single expression)
const add = (a, b) => a + b;

// Single parameter - parentheses optional
const square = n => n * n;

// No parameters - empty parentheses required
const greet = () => 'Hello, World!';

// Multi-line - must use { } and explicit return
const calculate = (a, b) => {
  const result = a * b;
  return result + 10;
};
```

Syntax Rules

(a, b) => 2+ params: always use ()

n => single param: () optional

() => no params: () required

=> expr single expr: implicit return

=> { ... } block body: explicit return

Arrow Functions + Array Methods

```
// Great with array methods
const nums = [1, 2, 3, 4, 5];

const evens = nums.filter(n => n % 2 === 0);
// [2, 4]
const doubled = nums.map(n => n * 2);
// [2,4,6,8,10]
const total = nums.reduce((sum, n) => sum + n, 0); // 15

// Chain them!
const result = nums
  .filter(n => n > 2)
  .map(n => n * n); // [9, 16, 25]
```

Arrow functions do NOT have their own 'this' — they inherit it from the enclosing scope

❌ ES5 — 'this' problem

```
// ES5 - this problem
function Timer() {
  this.seconds = 0;
  setInterval(function() {
    // 'this' here is the global object (or undefined in strict
    mode)
    // NOT the Timer instance!
    this.seconds++;          // ❌ Bug: wrong this!
    console.log(this.seconds); // NaN or error
  }, 1000);
}

// ES5 workaround: save this as self
function Timer() {
  this.seconds = 0;
  var self = this;          // workaround
  setInterval(function() {
    self.seconds++;          // ✅ works but ugly
    console.log(self.seconds);
  }, 1000);
}
```

✅ ES6 — Lexical this

```
// ES6 Arrow function - this is inherited
function Timer() {
  this.seconds = 0;
  setInterval(() => {
    // Arrow function inherits 'this' from Timer!
    this.seconds++;          // ✅ correct!
    console.log(this.seconds); // 1, 2, 3...
  }, 1000);
}

const t = new Timer();

// Vue.js real-world example:
// Inside a Vue component method:
fetchUsers() {
  fetch('/api/users')
    .then(res => res.json())
    .then(data => {
      this.users = data; // 'this' = Vue instance ✅
    });
}
```

⚠️ When NOT to use arrow functions: Object methods (needs own this), Event handlers that need this as the DOM element, Constructor functions (cannot use new with arrow functions)



Convert these ES5 functions to ES6 arrow functions

Questions

```
// Q1: Convert to arrow function
function double(n) {
  return n * 2;
}

// Q2: Convert to arrow function
var greet = function(name) {
  return "Hello, " + name + "!";
};

// Q3: Convert - use array methods
var nums = [1, 2, 3, 4, 5, 6];
var evenSquares = nums.filter(function(n) {
  return n % 2 === 0;
}).map(function(n) {
  return n * n;
});
console.log(evenSquares); // [4, 16, 36]

// Q4: Fix the 'this' bug
function Counter() {
  this.count = 0;
  setInterval(function() {
    this.count++; // Bug!
    console.log(this.count);
  }, 1000);
}
```



Solutions

04 Template Literals

String interpolation and multi-line strings

Use backticks (`) instead of quotes — enables interpolation and multi-line

❌ ES5 — Concatenation

```
// ES5 string concatenation
var name = 'Alice';
var age = 20;
var msg = "Hello, I'm " + name + " and I'm " + age + " years old.";
// Messy! Error-prone with many variables.

// Multi-line in ES5
var html = "<div>" +
    "  <h1>" + name + "</h1>" +
    "  <p>Age: " + age + "</p>" +
    "</div>";
```

✅ ES6 — Template Literal

```
// ES6 Template Literal — use backtick
const name = 'Alice';
const age = 20;
const msg = `Hello, I'm ${name} and I'm ${age} years old.`;
// Clean and readable!

// Multi-line — just press Enter!
const html = `
  <div>
    <h1>${name}</h1>
    <p>Age: ${age}</p>
  </div>
```

Expressions inside \${ } — any valid JS expression works

```
// Expressions in ${ }
const a = 5, b = 3;
console.log(`Sum: ${a + b}`); // Sum: 8
console.log(`Max: ${Math.max(a,b)}`); // Max: 5
console.log(`${a > b ? 'a' : 'b'} is bigger`); // a is bigger

// Function calls
const greet = name => `Hello, ${name.toUpperCase()}!`;
console.log(greet('alice')); // Hello, ALICE!

// Nested template (careful with readability)
const items = ['Apple', 'Banana'];
const list = `Items: ${items.map(i => `<li>${i}</li>`).join('')}`;
```

05 Destructuring

Unpack arrays and objects elegantly

Extract values from arrays or properties from objects into distinct variables

Array Destructuring

```
// Array Destructuring
const nums = [10, 20, 30, 40];

// Old way:
const a = nums[0], b = nums[1];

// ES6 Destructuring:
const [first, second, third] = nums;
console.log(first, second, third); // 10 20 30

// Skip elements with commas
const [x, , z] = nums; // x=10, z=30

// With rest
const [head, ...tail] = nums;
// head=10, tail=[20,30,40]

// Swap variables (no temp needed!)
let p = 1, q = 2;
[p, q] = [q, p];
console.log(p, q); // 2 1

// Default values
const [a2 = 0, b2 = 0, c2 = 99] = [1, 2];
console.log(c2); // 99 (default used)
```

Object Destructuring

```
// Object Destructuring
const user = { name: 'Bob', age: 25, role: 'admin' };

// Old way:
const name = user.name;
const age = user.age;

// ES6:
const { name, age, role } = user;
console.log(name, age, role); // Bob 25 admin

// Rename while destructuring
const { name: userName, age: userAge } = user;
console.log(userName); // Bob

// Default values
const { name: n, score = 100 } = user;
console.log(score); // 100 (default)

// Nested destructuring
const { address: { city } } = {
  name: 'Bob',
  address: { city: 'Bangkok', zip: '10100' }
};
console.log(city); // Bangkok

// In function parameters:
function showUser({ name, age }) {
  console.log(`${name} is ${age}`);
}

showUser(user); // Bob is 25
```



Use destructuring to simplify this code

Questions

```
const student = {
  id: 1001,
  name: 'Somchai',
  scores: [85, 90, 78, 92],
  address: {
    city: 'Bangkok',
    district: 'Pathum Wan'
  }
};

// Q1: Extract name and id from student
// (one line using destructuring)

// Q2: Extract first and last score
// from student.scores

// Q3: Extract city from student.address
// (using nested destructuring)

// Q4: Write a function printStudent(student)
// that uses parameter destructuring to print:
// "Somchai (1001) from Bangkok"
```



Solutions

06 Default, Rest & Spread

Flexible, expressive function signatures

Assign default values to function parameters — no more `a = a ||` default hacks

❌ ES5 Problem

```
// ES5 — manual default with ||
// BUG: 0 and '' are falsy!
function add(a, b) {
  a = a || 1; // if a=0, becomes 1! Bug!
  b = b || 2;
  return a + b;
}
console.log(add(0, 0)); // 1 + 2 = 3 ← Wrong!

// ES5 correct but verbose
function add(a, b) {
  a = (a !== undefined) ? a : 1;
  b = (b !== undefined) ? b : 2;
  return a + b;
}
```

✅ ES6 Default Parameters

```
// ES6 Default Parameters — clean!
function add(a = 1, b = 2) {
  return a + b;
}
console.log(add());           // 1 + 2 = 3
console.log(add(5));          // 5 + 2 = 7
console.log(add(0, 0));       // 0 + 0 = 0 ✅
console.log(add(undefined, 4)); // 1 + 4 = 5

// Defaults can be expressions!
function greet(name, msg = `Hello, ${name}!`) {
  return msg;
}
```

Rest Parameters (...rest)

```
// Rest Parameters — collect extra args into an array
// ES5 way (arguments object — ugly, not a real Array)
function sum() {
  return Array.prototype.slice.call(arguments).reduce((t,n) => t+n, 0);
}

// ES6 Rest — ...rest is a real Array!
function sum(first, ...rest) {
  console.log('first:', first); // 1
  console.log('rest:', rest);   // [2, 3, 4, 5]
  return rest.reduce((total, n) => total + n, first);
}
console.log(sum(1, 2, 3, 4, 5)); // 15

// rest must be the LAST parameter
function example(a, b, ...others) { /* OK */ }
```

Spread is the opposite of rest — it expands an array or object into individual items

Spread with Arrays

```
// Spread in function calls
const nums = [1, 2, 3];
console.log(Math.max(...nums)); // 3
console.log(Math.max(1, 2, 3)); // same!

// Copy arrays (shallow)
const a = [1, 2, 3];
const b = [...a];           // [1, 2, 3] — new array
b.push(4);
console.log(a); // [1, 2, 3] — unchanged

// Merge arrays
const c = [...a, ...b, 99]; // [1,2,3,1,2,3,4,99]

// Spread strings
console.log(...'hello'); // ['h','e','l','l','o']
```

Spread with Objects

```
// Spread with Objects (ES2018, widely used in Vue/Node)
const defaults = { theme: 'dark', lang: 'en', size: 14 };
const userPref = { lang: 'th', size: 16 };

// Merge — later keys override earlier ones
const config = { ...defaults, ...userPref };
// { theme: 'dark', lang: 'th', size: 16 }

// Copy object (shallow)
const original = { name: 'Alice', score: 90 };
const copy = { ...original, score: 95 }; // override score
// { name: 'Alice', score: 95 }

// In Vue.js — commonly used to spread Vuex state:
// ...mapState(['user', 'products'])
```



Rest vs Spread — same ... but different roles

```
// Rest vs Spread — Same syntax, different context
// REST: in function parameters → collects into array
function collect(...args) { return args; }
collect(1, 2, 3); // args = [1, 2, 3] ← REST

// SPREAD: in function calls or array/object literals → expands
const arr = [1, 2, 3];
console.log(...arr);           // 1 2 3 ← SPREAD
const copy = [...arr];         // [1, 2, 3] ← SPREAD
const obj = { ...{a:1} };      // {a: 1} ← SPREAD
```

07 Classes

Object-oriented programming with clean syntax

Classes are syntactic sugar over prototype-based inheritance — cleaner, not different

❌ ES5 — Prototype

```
// ES5 — Prototype-based (old way)
function Person(name, age) {
  this.name = name;
  this.age = age;
}
Person.prototype.greet = function() {
  return "Hi, I'm " + this.name;
};
Person.prototype.toString = function() {
  return this.name + " (" + this.age + ")";
};
var p = new Person("Alice", 20);
console.log(p.greet()); // Hi, I'm Alice

// Missing 'new' → silent disaster!
var p2 = Person("Bob", 22);
// name/age set on global object! 🌍
```

✅ ES6 — Class

```
// ES6 Class — clean syntax
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
  greet() {
    return `Hi, I'm ${this.name}`;
  }
  toString() {
    return `${this.name} (${this.age})`;
  }
}

const p = new Person('Alice', 20);
console.log(p.greet()); // Hi, I'm Alice

// Missing 'new' → TypeError (safe!)
// Person('Bob', 22) → TypeError!
```

Static Methods & Getters

```
// Static methods and getters
class MathHelper {
  static square(n) { return n * n; } // called on class, not instance
  static cube(n) { return n * n * n; }
}
console.log(MathHelper.square(4)); // 16

class Circle {
  constructor(radius) { this.radius = radius; }
  get area() { return Math.PI * this.radius ** 2; }
  get perimeter() { return 2 * Math.PI * this.radius; }
}
const c = new Circle(5);
console.log(c.area.toFixed(2)); // 78.54 (no parentheses needed!)
```

Build class hierarchies cleanly with extends. Use super to call the parent.

Inheritance Example

```
class Animal {
  constructor(name, sound) {
    this.name = name;
    this.sound = sound;
  }

  speak() {
    return `${this.name} says ${this.sound}!`;
  }

  toString() {
    return `Animal: ${this.name}`;
  }
}

class Dog extends Animal {
  constructor(name, breed) {
    super(name, 'Woof'); // must call super() first!
    this.breed = breed;
  }

  // Override parent method
  speak() {
    const parentMsg = super.speak(); // call parent
    return `${parentMsg} (tail wagging)`;
  }

  fetch(item) {
    return `${this.name} fetches the ${item}!`;
  }
}

const dog = new Dog('Rex', 'Labrador');
console.log(dog.speak()); // Rex says Woof! (tail wagging)
console.log(dog.fetch('ball')); // Rex fetches the ball!
console.log(dog instanceof Animal); // true
```

Inheritance Chain



Rules & Tips

```
// Key rules for extends / super:
// 1. Call super() before using 'this'
constructor(name) {
  super(name); // MUST be first
  this.extra = ''; // then 'this' is OK
}

// 2. super.method() calls parent version
speak() {
  return super.speak() + ' extra!';
}

// 3. instanceof checks the chain
dog instanceof Dog; // true
dog instanceof Animal; // true

// 4. You can't extend multiple classes
// Use mixins or composition instead
```



Build a simple Product & DiscountedProduct class hierarchy

Task

```
// Create a Product class with:
// - constructor(name, price)
// - get formattedPrice() → "$120.00"
// - describe() → "Product: Widget - $120.00"

// Then create DiscountedProduct extends Product:
// - constructor(name, price, discountPercent)
// - get discountedPrice()
//   → price * (1 - discountPercent / 100)
// - describe() → override to include discount info
//   "Product: Widget - $120.00 (20% off → $96.00)"

// Test:
const p = new Product('Widget', 120);
console.log(p.formattedPrice); // $120.00
console.log(p.describe());     // Product: Widget - $120.00

const d = new DiscountedProduct('Widget', 120, 20);
console.log(d.discountedPrice); // 96
console.log(d.describe());      // Product: Widget - $120.00
                                // (20% off → $96.00)
```

✓ Solution

08 Modules

import / export — organize your codebase

Split code across files. Each file is its own module with its own scope.

math.js — Exporting

```
// math.js — Named exports
export const PI = 3.14159;

export function add(a, b) {
  return a + b;
}

export function multiply(a, b) {
  return a * b;
}

// Or export all at end:
const subtract = (a, b) => a - b;
export { subtract };

// Default export (one per file)
export default class Calculator {
  add(a, b) { return a + b; }
```

app.js — Importing

```
// app.js — Importing
import Calculator from './math.js'; // default
import { PI, add, multiply } from './math.js';

// Rename on import
import { add as sum } from './math.js';

// Import everything as namespace
import * as Math2 from './math.js';
Math2.add(1, 2);

// Using imports
const calc = new Calculator();
console.log(calc.add(1, 2)); // 3
console.log(add(3, 4)); // 7
console.log(PI); // 3.14159
```

package.json

```
{
  "type": "module"
}
```

In Node.js and Vue.js context

```
// Node.js — use .mjs extension OR "type": "module" in package.json
// CommonJS (old Node): const fs = require('fs');
// ES Modules (Node ESM): import fs from 'fs';
// import { readFile } from 'fs';
```

```
// Vue.js — you'll see this everywhere:
import { ref, computed } from 'vue';
import MyComponent from './components/MyComponent.vue';
import { useRouter } from 'vue-router';
import { useStore } from 'vuex';
```

09 Promises & Async/Await

Handle asynchronous operations gracefully

A Promise represents a value that may be available now, in the future, or never

❌ ES5 — Callback Hell

```
// ES5 — Callback Hell
fetchUser(id, function(err, user) {
  if (err) { handleError(err); return; }
  fetchOrders(user.id, function(err, orders) {
    if (err) { handleError(err); return; }
    fetchDetails(orders[0], function(err, detail) {
      if (err) { handleError(err); return; }
      console.log(detail); // finally!
    });
  });
});
// Pyramid of doom! Hard to read and maintain.
```

✅ ES6 — Promise Chain

```
// ES6 Promise — flat chain
fetchUser(id)
  .then(user => fetchOrders(user.id))
  .then(orders => fetchDetails(orders[0]))
  .then(detail => console.log(detail))
  .catch(err => handleError(err)); // one error handler!

// Create a Promise
function delay(ms) {
  return new Promise((resolve, reject) => {
    setTimeout(() => resolve('done!'), ms);
  });
}
```

async / await — Write async code that reads like sync

```
// async/await — ES2017, syntactic sugar over Promises
async function loadData() {
  try {
    const user = await fetchUser(id); // wait for promise
    const orders = await fetchOrders(user.id);
    const detail = await fetchDetails(orders[0]);
    console.log(detail);
  } catch (err) {
    handleError(err);
  }
}
loadData();

// In Vue.js component — fetching from Node.js API:
async fetchProducts() {
  try {
    const response = await fetch('/api/products');
    const data = await response.json();
    this.products = data;
  } catch (err) {
    this.error = err.message;
  }
}
```



Convert callback-based code to async/await

Questions

```
// Given these Promise-based functions:
function getUser(id) {
  return new Promise(resolve => {
    setTimeout(() => resolve({ id, name: 'Alice' }), 300);
  });
}

function getPosts(userId) {
  return new Promise(resolve => {
    setTimeout(() => resolve([
      { id: 1, title: 'ES6 is great' },
      { id: 2, title: 'Vue.js tips' }
    ]), 200);
  });
}

// Q1: Use .then() chain to:
//   - get user with id=1
//   - then get their posts
//   - then log each post title

// Q2: Rewrite Q1 using async/await
//   with proper try/catch error handling
```



Solutions

10 New Built-ins

Map, Set, Symbol & Object improvements

Map: key-value pairs with any key type. Set: unique values only.

Map

```
// Map - like an object but better keys!
const map = new Map();
map.set('name', 'Alice');
map.set(42, 'the answer');
map.set({ id: 1 }, 'object key!'); // ← objects as keys!

map.get('name');    // 'Alice'
map.has('name');    // true
map.size;           // 3
map.delete('name'); // removes it

// Iterate Map
for (const [key, value] of map) {
  console.log(key, '→', value);
}

// Initialize from array of pairs
const config = new Map([
  ['host', 'localhost'],
  ['port', 3000],
  ['debug', true]
]);
```

Set

```
// Set - collection of unique values
const set = new Set([1, 2, 3, 2, 1]);
console.log(set.size); // 3 (duplicates removed)
console.log([...set]); // [1, 2, 3]

set.add(4);
set.has(2);    // true
set.delete(1);

// Iterate Set
for (const item of set) {
  console.log(item); // 2, 3, 4
}

// Common use: remove duplicates from array!
const nums = [1, 2, 2, 3, 3, 3, 4];
const unique = [...new Set(nums)];
console.log(unique); // [1, 2, 3, 4]
```

Object Improvements

```
// Object.assign() - merge objects (ES6)
const defaults = { theme: 'light', lang: 'en' };
const user      = { lang: 'th', user: 'Alice' };
const merged    = Object.assign({}, defaults, user);
// { theme: 'light', lang: 'th', user: 'Alice' }

// Object.keys(), values(), entries()
const obj = { a: 1, b: 2, c: 3 };
Object.keys(obj);    // ['a', 'b', 'c']
Object.values(obj);   // [1, 2, 3]
Object.entries(obj);  // [['a', 1], ['b', 2], ['c', 3]]

// for...of with entries
for (const [k, v] of Object.entries(obj)) {
  console.log(`${k}: ${v}`);
}
```

Shorter, more expressive object syntax used everywhere in Vue.js and Node.js

Object Literal Features

```
// Property shorthand
const name = 'Alice';
const age  = 20;

// ES5:
var person = { name: name, age: age };

// ES6 - when key and variable name match:
const person = { name, age };
// → { name: 'Alice', age: 20 }

// Method shorthand
// ES5:
var obj = {
  greet: function() { return 'Hello!'; }
};

// ES6:
const obj = {
  greet() { return 'Hello!'; },
  async fetchData() { /* ... */ } // async method too!
};

// Computed property keys
const field = 'score';
const player = {
  name: 'Bob',
  [field]: 100,           // computed: score: 100
  [`${field}Max`]: 200 // computed: scoreMax: 200
};
console.log(player.score);    // 100
console.log(player.scoreMax); // 200
```

Vue.js & Node.js Usage

```
// Real-world Vue.js examples:
import { mapState, mapMutations } from 'vuex';

export default {
  name: 'MyComponent',

  // Computed shorthand - method shorthand everywhere
  computed: {
    ...mapState(['user', 'products']), // spread!
    totalItems() {
      return this.products.length;
    }
  },

  methods: {
    ...mapMutations(['setUser']),
    async loadData() {
      const res = await fetch('/api/data');
      const data = await res.json();
      this.setUser(data.user); // mutation
    }
  }
};

// Node.js Express route example:
const userId = req.params.id;
const name = req.body.name;

// Shorthand in response:
res.json({ userId, name, success: true });
```

All the key ES6 features in one place

let / const

```
let x = 1; const PI = 3.14;
```

Arrow Function

```
const fn = (a, b) => a + b;
```

Template Literal

```
`Hello, ${name}!`
```

Destructuring

```
const {a} = obj; const [x] = arr;
```

Default Params

```
function f(a=1, b=2) {}
```

Rest / Spread

```
function f(...args) {} [...arr]
```

Class

```
class Dog extends Animal { }
```

import/export

```
import {fn} from './mod.js';
```

You now know ES6! 🎉

What we covered today:

- ✓ let & const — block scoping, no more var
- ✓ Arrow functions — shorter + lexical this
- ✓ Template literals — clean string interpolation
- ✓ Destructuring — unpack arrays and objects
- ✓ Default, rest & spread — flexible functions
- ✓ Classes & inheritance — clean OOP
- ✓ Modules — import/export organization
- ✓ Promises & async/await — async programming
- ✓ Map, Set, extended object literals

Next Lecture: Node.js & Express.js